

# Champion Document — Feature 3

## Enemy AI & Encounter Spawning

Name: \_\_\_\_\_ Mark: \_\_\_\_ / 30

### 1. Introduction

My feature populates rooms with enemies and controls their basic combat behavior. *SpawnEncounter(Room room, int difficultyTier)* uses room type and difficulty information from Feature 2 to select and spawn the appropriate enemy types. Each active enemy runs a patrol → chase → attack decision cycle. When an enemy is defeated, the feature raises *OnEnemyDefeated* with the defeated Enemy object.

The behavior design uses the **Template Method** pattern. *EnemyBehavior* is an abstract, non-MonoBehaviour base class. *RunCycle()* fixes the patrol → chase → attack skeleton, while *MeleeEnemy*, *RangedEnemy*, and *TurretEnemy* override *PerformAttack()*. Removing the override point would make every enemy use the base melee-lunge behavior, which is visibly wrong for ranged and turret enemies.

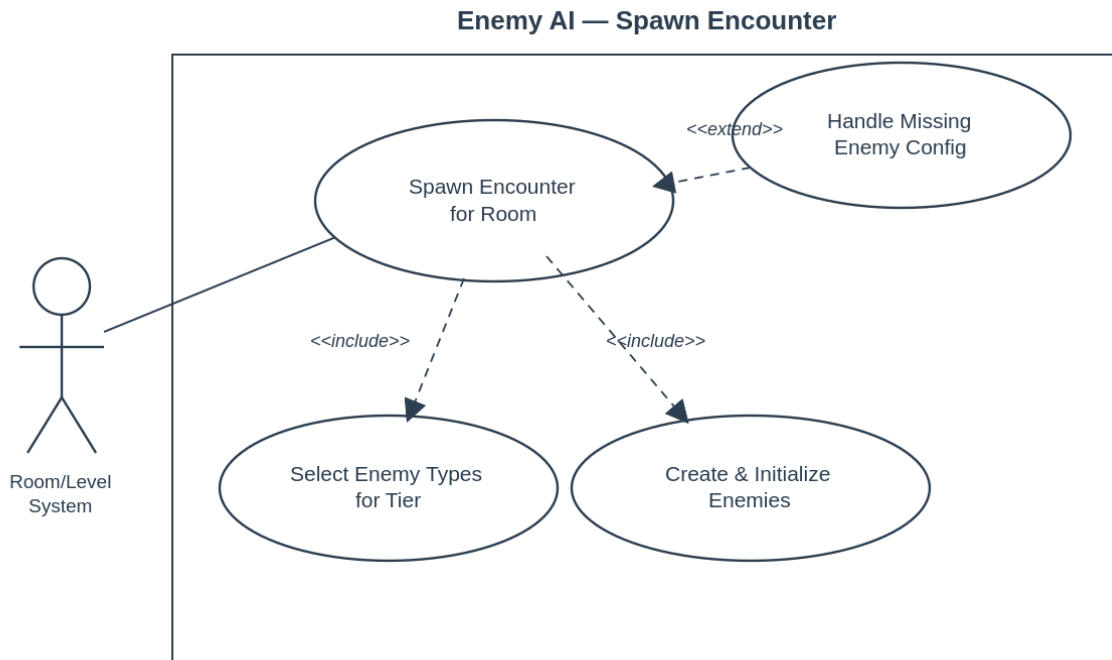
GRASP: **Pure Fabrication** — *EncounterSpawner* doesn't map to a game “thing”; it exists to keep spawning logic out of *Room*. **Indirection** — the spawner mediates between the dungeon map and the enemies it creates.

#### This feature does NOT:

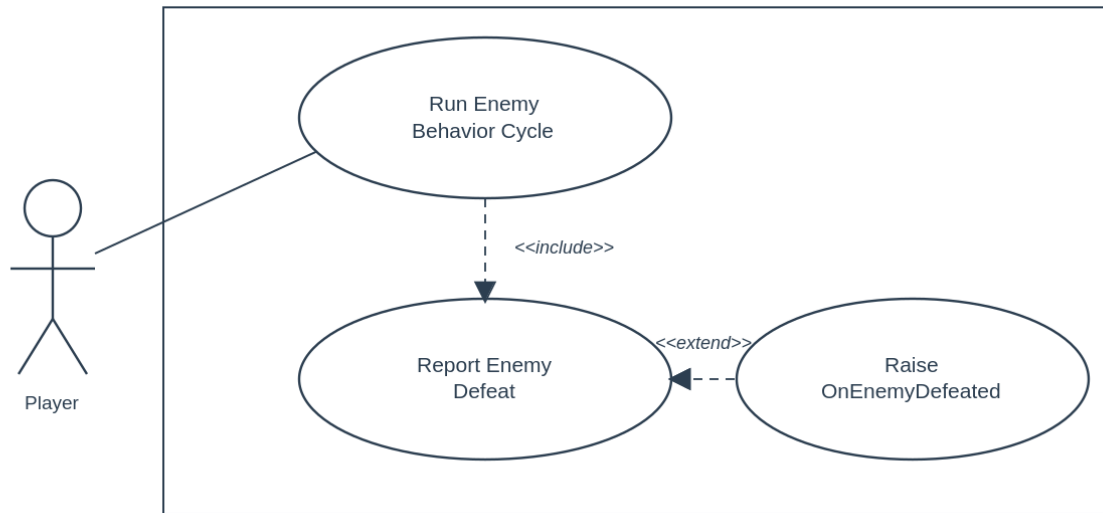
- decide room layout or generate the room; that comes from Feature 2.
- own the player position; it reads it through Feature 7 *GetPosition()*.
- resolve damage itself; damage is consumed through Feature 7 *ApplyDamage()*.
- define unrelated player systems, UI, or overall game progression.

### 2. Use Case Diagrams with Scenario

#### Use Case Diagrams



## Enemy AI — Behavior & Defeat



Arrow conventions follow the Pearson UML legend: *Communicates* is a solid line with no arrowhead. <<include>> and <<extend>> are dashed with a filled arrowhead — include points to the common (included) use case; extend points from the exception case to the basic use case.

### Scenario 1 — Spawn Encounter for Room

**Actors:** Room/Level system (Feature 2), Enemy Runtime

**Preconditions:** Room is valid; room type data is available; difficultyTier is an accepted integer; enemy types/prefabs are available.

#### Basic sequence:

1. The room system calls SpawnEncounter(Room room, int difficultyTier).
2. EncounterSpawner reads room type and difficulty tier and determines the enemy types/count allowed for that tier.
3. EncounterSpawner creates the selected enemy instances and initializes their EnemyBehavior objects.
4. Each active enemy is registered in runtime enemy state.
5. The encounter is active and each enemy can begin RunCycle().

#### Exceptions:

- Step 1a: The Room reference is null or invalid. The request is rejected and no enemy instances are created.
- Step 2a: No enemy set is configured for the room type/tier. No encounter is created and the configuration failure is logged/reported.
- Step 3a: An enemy instance cannot be created. That enemy is skipped; already-created valid enemies remain active.

**Postconditions:** Every successfully spawned enemy has an initialized behavior object and is available for behavior updates.

**Priority:** 1 (must have) **ID:** SE1

### Scenario 2 — Run Enemy Behavior and Report Defeat

**Actors:** Player (Feature 7), Combat/Player damage interface (Feature 7), Enemy Runtime

**Preconditions:** Enemy is active; EnemyBehavior is initialized; Feature 7 can provide player position and consume damage.

#### Basic sequence:

6. RunCycle() begins for an active EnemyBehavior.

7. The behavior reads the player position using Feature 7 `GetPosition()` and decides whether to remain in PATROL or move toward CHASE.
8. If the player meets the attack condition, the behavior enters ATTACK.
9. `RunCycle()` calls the subtype `PerformAttack()` override exactly once for that cycle.
10. The attack uses Feature 7 `ApplyDamage()` as defined by the implementation.
11. If HP reaches 0, the feature removes/deactivates the enemy from active behavior and raises `OnEnemyDefeated(Enemy)`.
12. If the enemy is not defeated, the next behavior cycle begins.

**Exceptions:**

- Step 2a: Player is outside the chase/aggro condition. Enemy remains in or returns to PATROL and does not attack.
- Step 3a: Player is not within attack range/condition. Enemy remains in CHASE and does not call `PerformAttack()` for that cycle.
- Step 5a: Feature 7 does not accept the damage request. The cycle ends without treating the enemy as defeated.
- Step 6a: HP remains above 0. `OnEnemyDefeated` is not raised and the enemy continues.

**Postconditions:** Enemy is active in PATROL/CHASE/ATTACK or has been defeated and reported.

**Priority:** 1 (must have) **ID:** EE1

### 3. Interface Contract

Every public input/output/event below is typed and testable.

| Item            | Type                   | Input / Trigger                          | Observable result                                       | Pass condition                                                                                  |
|-----------------|------------------------|------------------------------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| SpawnEncounter  | void method            | Room room; int difficultyTier            | Creates 0+ configured enemies                           | For a valid room/tier, every created enemy is allowed for that tier; an invalid room creates 0. |
| OnEnemyDefeated | event<br>Action<Enemy> | Raised when an active enemy reaches HP 0 | Event payload is the defeated Enemy                     | Exactly one event per defeated Enemy.                                                           |
| RunCycle        | void method            | No arguments                             | Updates state and may call <code>PerformAttack()</code> | One call performs one decision cycle; the attack hook fires at most once.                       |
| PerformAttack   | protected virtual void | No arguments; overridden by subtype      | Subtype-specific attack                                 | Each subtype executes its own override when attack conditions are met.                          |

### Dependencies

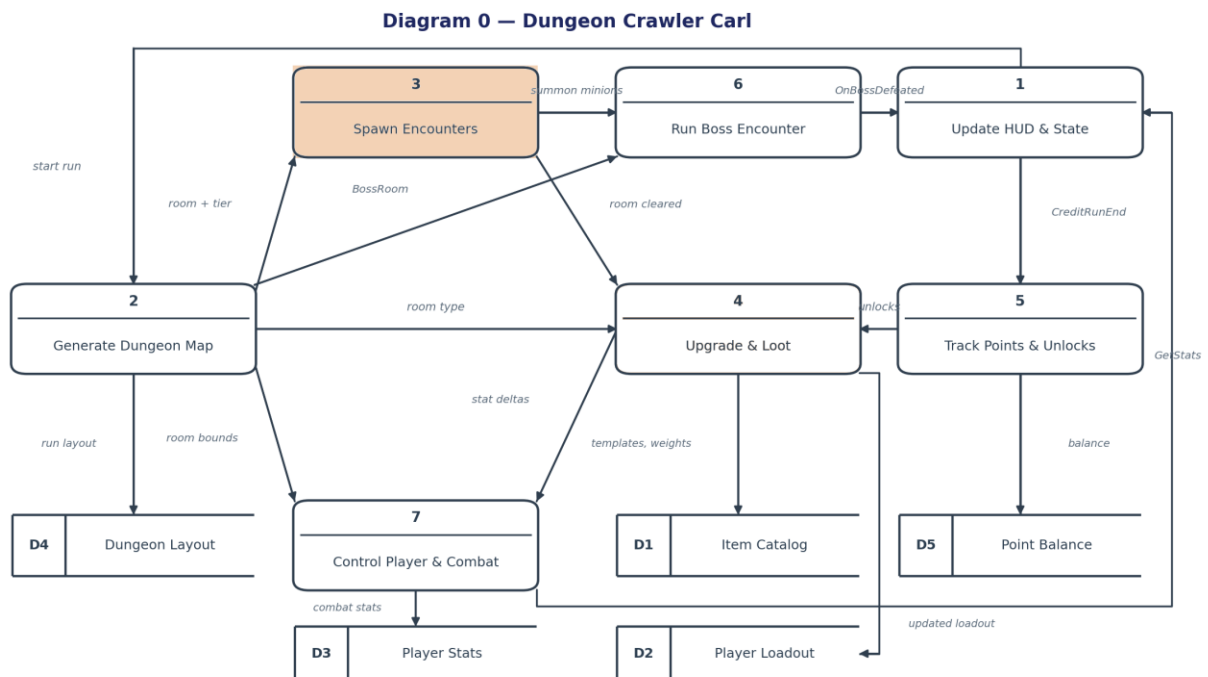
| Dependency                           | Direction     | Typed contract                                                |
|--------------------------------------|---------------|---------------------------------------------------------------|
| Feature 2 Room                       | Read          | Room supplies room type; difficultyTier is int.               |
| Feature 7 <code>GetPosition()</code> | Read          | Returns player position in the project's position type.       |
| Feature 7 <code>ApplyDamage()</code> | Call          | Consumes the project-defined damage request/target interface. |
| Enemy                                | Event payload | <code>OnEnemyDefeated</code> uses Action<Enemy>.              |

## Internal Design

- EncounterSpawner — Pure Fabrication; keeps spawning logic out of Room.
- EnemyBehavior — abstract, non-MonoBehaviour base class using Template Method.
- MeleeEnemy, RangedEnemy, TurretEnemy — override PerformAttack() with distinct attacks.
- Indirection — EncounterSpawner mediates between room/map data and enemy creation.

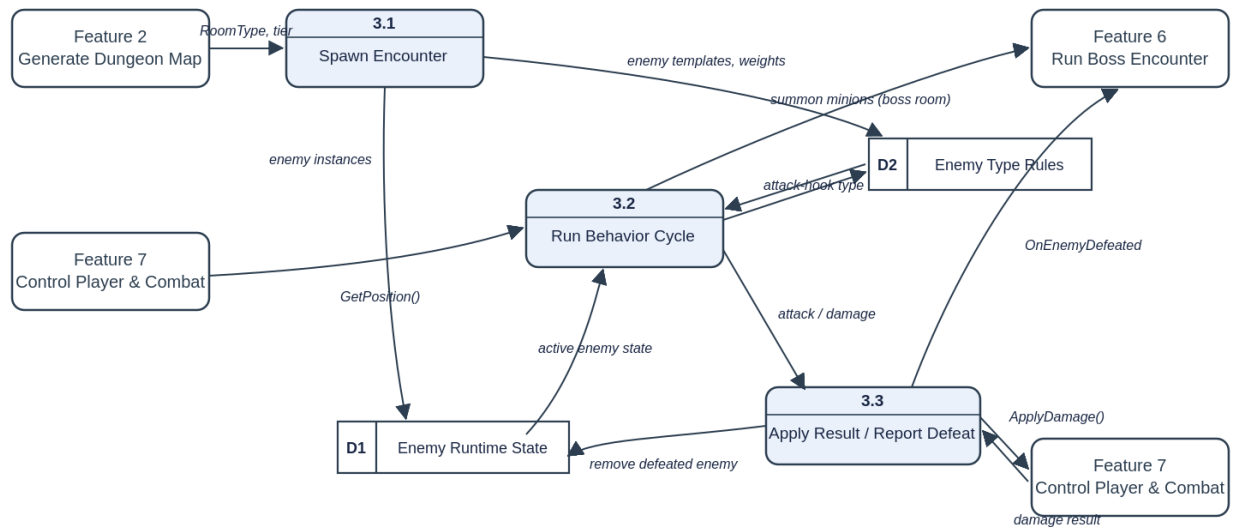
## 4. Data Flow Diagrams

Diagram 0 shows the whole game: one numbered process for each feature in the Feature Specifications (1 HUD & game state, 2 dungeon map, 3 encounters, 4 upgrade & loot, 5 point economy, 6 boss, 7 player combat). **Process 3** is highlighted as this Champion's feature. Processes are Gane–Sarson rounded rectangles with a numbered band; data stores are open rectangles with a left-hand identifier. Every process has at least one incoming and one outgoing flow (no black holes or miracles). D4 Dungeon Layout, D3 Player Stats, D5 Point Balance, and D1/D2 (Item Catalog / Player Loadout) belong to other features; my own zoom introduces D1 Enemy Runtime State and D2 Enemy Type Rules, local to Process 3.



I then zoom into Process 3 and give a decision-tree description for primitive process 3.2.

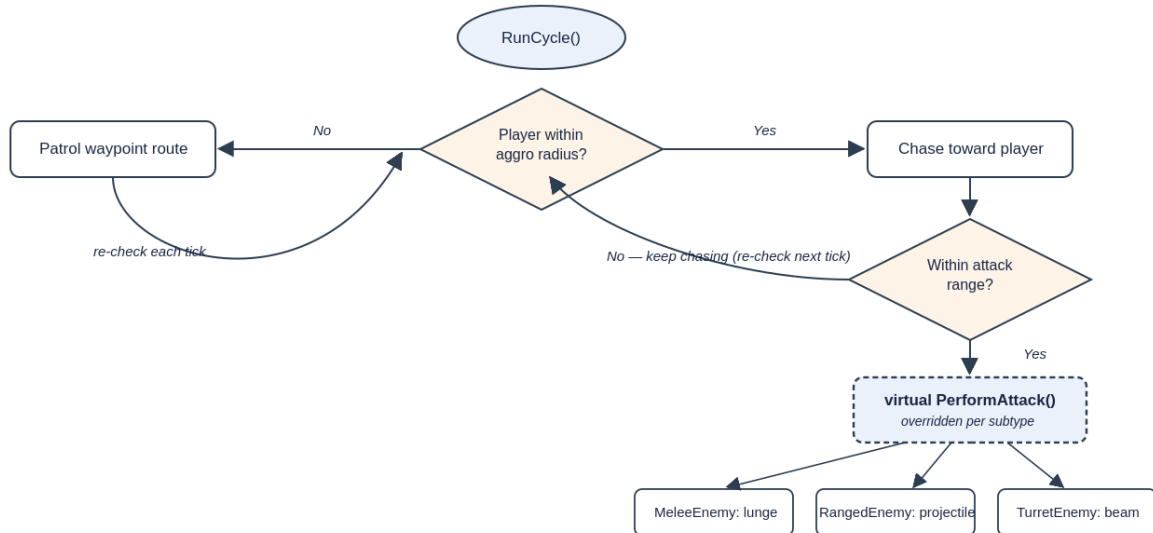
**Diagram 3 — Enemy AI & Encounter Spawning (zoom of Process 3)**



Process notes: **3.1** receives RoomType and difficultyTier and produces enemy instances, reading enemy templates/weights from D2. **3.2** reads player position and D1's active enemy state, decides PATROL/CHASE/ATTACK, and invokes the subtype *PerformAttack()* hook. **3.3** consumes the damage result, updates D1, and emits OnEnemyDefeated when HP reaches 0.

## Process Description — 3.2 Run Behavior Cycle

**Process Description — 3.2 Run Behavior Cycle (Template Method)**



3.2 is a primitive: it does not decompose further. It is the Template Method skeleton — the patrol/chase/attack-range decision logic is fixed and identical for every enemy, while the shaded PerformAttack() step is a virtual hook that each subtype (MeleeEnemy, RangedEnemy, TurretEnemy) overrides independently. Completion criteria from the spec: each enemy type spawns correctly per room tier, executes distinct attack behavior, and reports defeat events.

## Verification / Acceptance Tests

| Test                 | Procedure                                                              | Pass condition                                                      |
|----------------------|------------------------------------------------------------------------|---------------------------------------------------------------------|
| Spawn by tier        | Call SpawnEncounter for every configured room type and supported tier. | Every spawned enemy is an allowed subtype for that room/tier.       |
| RunCycle transitions | Use a mock player-position provider with scripted positions.           | PATROL → CHASE → ATTACK occurs only under the defined conditions.   |
| Subtype attack hooks | Run one attack-eligible cycle for each enemy subtype.                  | Each subtype override fires exactly once; no fallback melee attack. |
| Defeat event         | Damage a known-HP enemy until HP reaches 0.                            | OnEnemyDefeated fires exactly once with that Enemy.                 |
| No false defeat      | Apply damage that leaves HP > 0.                                       | No OnEnemyDefeated event is raised.                                 |

## 5. Timeline (Feature 3 only)

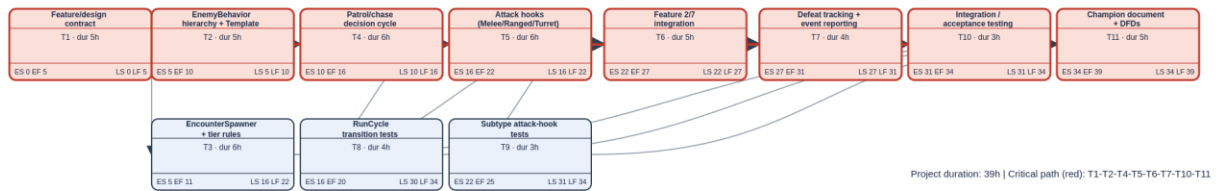
This schedule is my Feature 3 work only — not the seven-feature project calendar. Required estimate: 52 hours total — Design 9 h, Coding 28 h, Testing 10 h, Documentation 5 h. Because several coding and testing tasks run in parallel, calendar time on the critical path is 39 hours.

| Work item                                    | Hours | Predecessor(s) | Category      |
|----------------------------------------------|-------|----------------|---------------|
| 1. Feature/design contract                   | 5     | —              | Design        |
| 2. EnemyBehavior hierarchy + Template Method | 5     | 1              | Design/Coding |
| 3. EncounterSpawner + tier rules             | 6     | 1              | Design/Coding |
| 4. Patrol/chase decision cycle               | 6     | 2              | Coding        |
| 5. Melee/Ranged/Turret attack hooks          | 6     | 2, 4           | Coding        |
| 6. Feature 2 / Feature 7 integration         | 5     | 3, 4, 5        | Coding        |
| 7. Defeat tracking + event reporting         | 4     | 5, 6           | Coding        |
| 8. RunCycle transition tests                 | 4     | 4              | Testing       |
| 9. Subtype attack-hook tests                 | 3     | 5              | Testing       |
| 10. Integration/acceptance testing           | 3     | 6, 7           | Testing       |
| 11. Champion document + DFDs                 | 5     | 1–10           | Documentation |

Total: 52 hours. Planned category totals: Design 9 h; Coding 28 h; Testing 10 h; Documentation 5 h. The estimate excludes pathfinding, loot systems, animation art, and room-layout generation because they are outside the stated feature requirements.

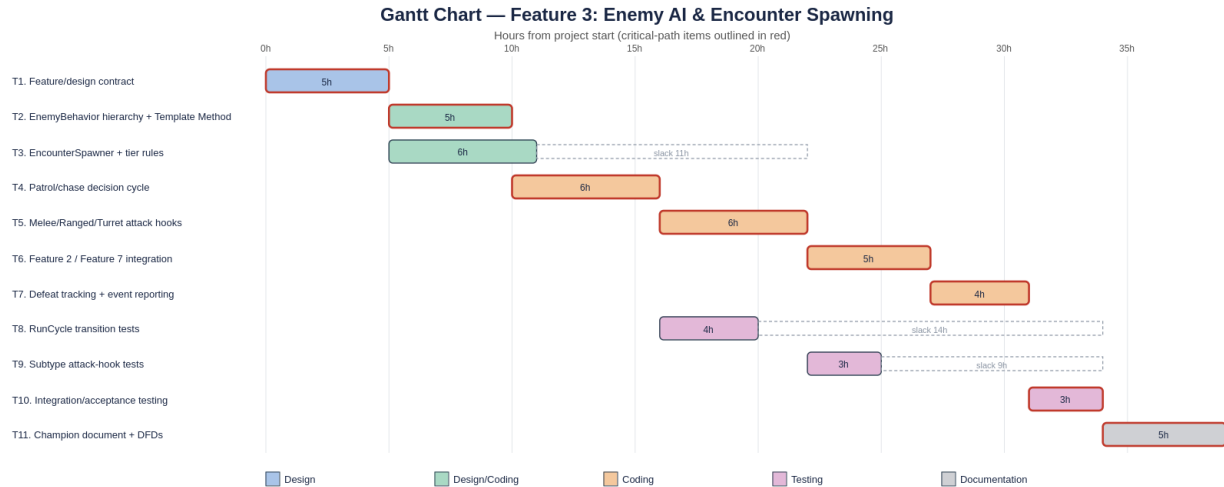
## PERT diagram (Feature 3 tasks)

## PERT / Activity Network — Feature 3 (critical path highlighted)



PERT node numbers are the Feature 3 work items in the table above, not Diagram 0's feature IDs. The critical path is 1 → 2 → 4 → 5 → 6 → 7 → 10 → 11, totaling 39 hours of calendar time. Task 3 (EncounterSpawner + tier rules) carries 11 hours of slack, task 8 (RunCycle tests) carries 14, and task 9 (subtype attack-hook tests) carries 9 — all three can slip without moving my finish, but the EnemyBehavior/Template Method chain cannot.

## Gantt timeline (Feature 3 only)



## AI Question Log — Summary

I used Claude to draft the Champion document from the course template, the sample Champion, the Feature Specifications, and the assigned diagram-line convention. I told it Diagram 0 had to stay the whole-team diagram with only Process 3 highlighted, and the rest had to focus on Feature 3 with correct <<include>>/<<extend>> arrows, at least one of each, and exceptions tied to numbered steps.

Claude caught that my <<include>> arrows were missing arrowheads (my <<extend>> arrows were already right), rebuilt my DFD into proper Gane–Sarson notation, and added a Process Description diagram for 3.2. It also found a 1-hour error in my timeline totals, fixed it, and used the corrected numbers to compute the PERT critical path and Gantt chart. The acceptance-test bounds are proposed numbers we added to make the spec testable.